**Professores:**

Celso Giusti

Daniel Manoel Filho

Isadora Dias Afonso

Marlon Palata Fanger Rodrigues

# DOM – DOCUMENT OBJECT MODEL

SENAI

DOM - DOCUMENT OBJECT MODEL

# O que é o DOM?

Antes de manipular o site, precisamos entender onde o JavaScript "mora" no navegador.

- **window**: É o objeto global. Representa a janela do navegador. Tudo o que acontece no navegador (abrir abas, histórico, alertas) está aqui.
- **document**: É um objeto que faz parte do **window**. Ele representa especificamente o **conteúdo HTML** da página atual.

**Onde usar:** Usamos **window** para comandos da janela (ex: **window.alert()**). Usamos **document** para acessar qualquer tag do site (ex: **document.querySelector()**).

**Por que usar:** Porque o JavaScript precisa saber se você quer falar com o "Navegador" (window) ou com a "Página" (document).

DOM - DOCUMENT OBJECT MODEL

# O que é o DOM?

A sigla **DOM** significa *Document Object Model* (Modelo de Objeto do Documento). Quando o navegador carrega seu HTML, ele não o vê apenas como um texto, mas como um **Objeto Global** chamado **window**. Dentro dele, existe o **document**, que é o mapa de toda a sua página.

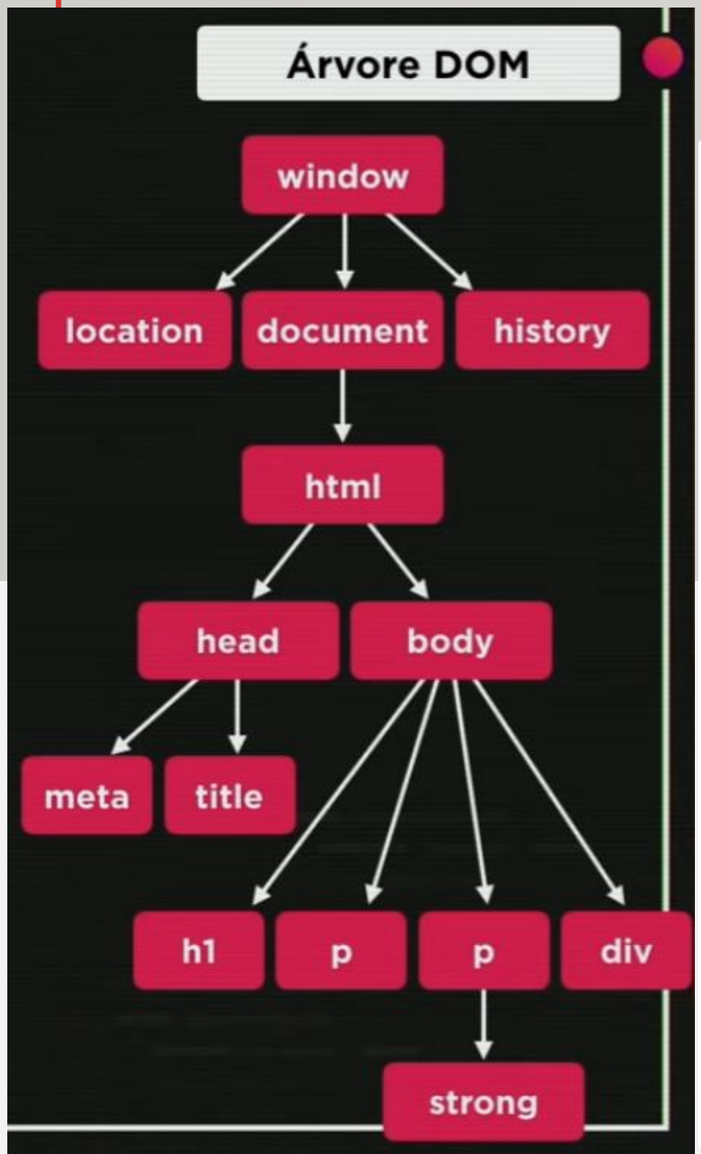
- **Document:** É o seu arquivo HTML.
- **Object:** Cada tag (**‹h1›, ‹div›, ‹p›**) vira um objeto dentro do JavaScript.
- **Model:** É a forma como esses objetos são organizados.

**Modelo em Árvore:** O DOM organiza as tags em uma hierarquia de "Pai e Filho". Se você tem um **‹div›** com um **‹h1›** dentro, a **div** é o **Parent Node** (Pai) e o **h1** é o **Child Node** (Filho).

**Dinamicidade:** Sem o DOM, o site é um panfleto parado. Com o DOM, o JS pode criar, deletar e transformar qualquer elemento enquanto o usuário navega.

DOM - DOCUMENT OBJECT MODEL

# Analogia da Árvore Genealógica DOM




**Acesso ao link do curso de JavaScript do Guanabara:**

https://www.youtube.com/watch?v=1-w1RfGIov4&list=PLHz_AreHm4dlsK3Nr9GVvXCbpQyHQl1o1

DOM - DOCUMENT OBJECT MODEL

# Seletores "Old School" (Antigo)

Antigamente, o JavaScript tinha ferramentas muito específicas e limitadas para encontrar elementos. Elas eram "rígidas" porque você precisava saber exatamente se estava buscando por um ID ou por uma Classe.

- **getElementById('id')**: Busca um único elemento através do seu ID único. É extremamente rápido, mas só serve para IDs.
- **getElementsByClassName('classe')**: Busca todos os elementos que possuem aquela classe e os devolve em uma lista (HTMLCollection).
- **getElementsByTagName('tag')**: Busca todos os elementos por nome da tag (ex: todos os p).

**Onde usar:** Quando você tem certeza absoluta de que o elemento tem um ID único ou uma classe específica e quer performance máxima em **navegadores antigos.**

DOM - DOCUMENT OBJECT MODEL

# Seletores "Old School" EXEMPLO (Antigo)

```
// Buscando o cabeçalho pelo ID único
const topo = document.getElementById('topo-loja');

// Buscando todos os cards de uma vez pela classe
const listaCards = document.getElementsByClassName('card');

// Buscando todos os links do menu
const links = document.getElementsByTagName('a');

console.log("1. O elemento topo:", topo);
console.log("2. O primeiro card da lista:", listaCards[0]);
console.log("3. A lista completa de links:", links);
console.log("4. O endereço (URL) do primeiro link:", links[0].href);
```

DOM - DOCUMENT OBJECT MODEL

# Seletores "New School": querySelector - (Moderno)

Com a evolução da web, surgiu o **querySelector** e o **querySelectorAll**. Eles unificaram a busca. Em vez de decorar vários comandos, você usa a **mesma sintaxe do CSS**.

- **Vantagem:** Se você sabe selecionar no CSS com #, . ou seletores compostos, você já sabe usar esses comandos.
- **Diferença Crucial:** O **querySelector** é "preguiçoso": se ele encontrar dez elementos com a mesma classe, ele pegará **apenas o primeiro** e parará a busca.

**Exemplo:**

```
// Busca pelo ID (usa # como no CSS)
const header = document.querySelector('#topo');

// Busca pela Classe (usa . como no CSS)
const btn = document.querySelector('.btn-enviar');

// Busca Combinada: O h3 que está dentro da div de ID 'card-nhoque'
const tituloPrato = document.querySelector('#card-nhoque h3');
```

DOM - DOCUMENT OBJECT MODEL

# Seletores "New School": querySelectorAll - (Moderno)

Similar ao **querySelector**, porém ele retorna **todos** os elementos que correspondem ao seletor informado.

**O Porquê:** Frequentemente precisamos aplicar uma regra a vários elementos de uma vez (ex: todos os botões de "comprar" da vitrine). O resultado é uma **NodeList** (uma lista de nós).

**Exemplo:**

```
// Seleciona TODOS os elementos que possuem a classe 'card'
const todosCards = document.querySelectorAll('.card');

// Mostra a quantidade de elementos encontrados
console.log(todosCards.length);
```

DOM - DOCUMENT OBJECT MODEL

# Manipulação de Texto: innerHTML vs textContent

São propriedades usadas para ler ou alterar o que está escrito dentro de um elemento(TAG):

1. **textContent**: Altera apenas o texto puro. É mais performático e seguro, pois o navegador não tenta "entender" o que você escreveu como código.

2. **innerHTML**: Altera o conteúdo permitindo o uso de tags HTML. Se você escrever **‹strong›Ok‹/strong›**, ele deixará a palavra em negrito.

- **Segurança:** Nunca use **innerHTML** para exibir algo que o usuário digitou, pois ele pode injetar códigos maliciosos (XSS).

DOM - DOCUMENT OBJECT MODEL

# Manipulação de Texto

# EXEMPLO

```
const aviso = document.querySelector('#mensagem');

// Altera apenas o texto. Se colocar tags, elas aparecem como texto comum.
aviso.textContent = "Olá, bem-vindo!";

// Altera o conteúdo e o navegador interpreta o <strong> como negrito
aviso.innerHTML = "Status: <strong>Confirmado</strong>";
```

DOM - DOCUMENT OBJECT MODEL

# Manipulando Atributos - (Antigo) (getAttribute e setAttribute)

Atributos são as propriedades "extras" das tags: o **src** da imagem, o **href** de um link ou o **alt** de acessibilidade. O JavaScript pode ler (**getAttribute**) ou alterar (**setAttribute**) esses valores a qualquer momento, permitindo, por exemplo, trocar a foto de um prato dinamicamente.

**Onde usar:** Trocar a foto de um prato, mudar o link de um botão ou trocar o "tipo" de um input de senha para texto.

DOM - DOCUMENT OBJECT MODEL

# Manipulando Atributos EXEMPLO (Antigo)

Vamos simular uma troca de imagem. Quando o prato acabar, vamos trocar a foto por uma imagem de "Esgotado".

```
const foto = document.querySelector('#foto-prato');

// COMO: Lendo o caminho atual da imagem
const caminhoAtual = foto.getAttribute('src');

// COMO: Trocando a imagem para "esgotado" dinamicamente
foto.setAttribute('src', 'img/esgotado.png');

// COMO: Alterando o texto de acessibilidade
foto.setAttribute('alt', 'Prato não disponível');
```

DOM - DOCUMENT OBJECT MODEL

# Manipulando Atributos - (Moderno)

Tratamos o elemento como um **Objeto JavaScript**. Se o objeto tem uma característica, mudamos ela direto.

- **Sintaxe:**

  **elemento.propriedade = 'valor'**

- **Vantagem:** Código limpo, leitura rápida e performance superior.

| O que você quer fazer | Jeito setAttribute (Antigo) | Jeito Direto (Moderno) |
|---|---|---|
| **Mudar uma Imagem** | img.setAttribute('src', 'foto.png') | img.src = 'foto.png' |
| **Mudar o Alt** | img.setAttribute('alt', 'Desc') | img.alt = 'Desc' |
| **Pegar um Link** | link.getAttribute('href') | link.href |
| **Desabilitar Botão** | btn.setAttribute('disabled', true) | btn.disabled = true |

DOM - DOCUMENT OBJECT MODEL

# Manipulando Atributos EXEMPLO (Moderno)

Mesmo exemplo da manipulação de atributos antiga, porém agora bem mais simplificado. O resultado será o mesmo.

```
const foto = document.querySelector('#foto-prato');

// MODERNO: Lendo o caminho atual (Direto na propriedade)
const caminhoAtual = foto.src;

// MODERNO: Trocando a imagem (Igual a mudar o valor de uma variável)
foto.src = 'img/esgotado.png';

// MODERNO: Alterando o texto de acessibilidade
foto.alt = 'Prato não disponível';
```

DOM - DOCUMENT OBJECT MODEL

# Captura de Dados (.value) e Conversão (Number)

A porta de entrada para o Back-end. O **.value** pega o que foi digitado.

**Onde usar:** Em formulários de cadastro, login e calculadoras.

**ATENÇÃO,** no JS, **todo input devolve texto**. Para fazer conta, você deve converter para número, senão "1+1" vira "11".

**Exemplo:**

```
const campoNome = document.querySelector('#nome');
const campoPreco = document.querySelector('#preco');

function processar() {
   const nome = campoNome.value; // Captura o texto (ex: "Pizza")
   const preco = Number(campoPreco.value); // CONVERSÃO: Texto para Número

   console.log(`Produto: ${nome} | Preço: ${preco}`);
}
```

DOM - DOCUMENT OBJECT MODEL

# Manipulação de Estilos (.style) - (Antigo)

Acesso direto às propriedades CSS da tag via JavaScript.

**Onde usar:** Quando você precisa de uma mudança rápida e única (ex: mudar a cor de um erro ou a posição de um ícone).

**Por que usar:** É a forma mais direta de mexer no visual, mas polui o HTML com estilos "inline".

**Exemplo :**

```
const displayPreco = document.querySelector('.preco-total');

// COMO: Acessamos a propriedade style, seguida do nome da regra CSS
// NOTA: No JS usamos camelCase (backgroundColor) em vez de hifens (background-color)
displayPreco.style.color = "red";
displayPreco.style.fontSize = "2rem";
displayPreco.style.fontWeight = "bold";
```

DOM - DOCUMENT OBJECT MODEL

# Manipulação Classes (classList) - (Moderno)

Embora possamos mudar o CSS direto com **.style.color = "red"**, isso gera o chamado "estilo inline", que é difícil de manter. A boa prática profissional é o **classList**. Você cria a classe no arquivo CSS e usa o JS apenas para "colar" ou "descolar" essa classe do elemento.

- **.add():** Adiciona a classe.
- **.remove():** Remove.
- **.toggle():** Se tem a classe, tira; se não tem, coloca.

**Onde usar:** Feedback de erro (borda vermelha), Dark Mode ou esconder elementos. **Por que usar: classList** é melhor que **.style** porque mantém o design no arquivo CSS e o JS apenas "ativa" o estilo.

DOM - DOCUMENT OBJECT MODEL

# classList

# EXEMPLO - (Moderno)

```
const box = document.querySelector('.mensagem');

// ONDE: Aplica o estilo de erro definido no CSS
box.classList.add('erro-ativo');

// ONDE: Alterna entre Modo Claro e Escuro
document.body.classList.toggle('dark-mode');

// COMO: Usando .style apenas para ajustes dinâmicos e específicos
box.style.marginTop = "20px";
```

DOM - DOCUMENT OBJECT MODEL

# Resumo

✅ **window vs document:** O **window** é o navegador, o **document** é a página.

✅ **Seletores: querySelector** é o padrão moderno que usa sintaxe CSS.

✅ **Texto: textContent** é para texto seguro, **innerHTML** para injetar HTML.

✅ **Manipulação de Atributos:** altera links e imagens dinamicamente.

✅ **Estilo Cru: .style** muda o CSS direto na tag (inline).

✅ **Estilo Profissional: classList** gerencia classes prontas do seu arquivo CSS.

ATIVIDADE

# Portal EcoCycle

1

## O Ambiente Global (Window vs Document)

**Contexto: Portal EcoCycle (Notícias de Ecologia)**

Crie um arquivo **eco.js** na pasta **SCRIPTS**

No início do seu arquivo **eco.js**, utilize o **console.log** para exibir no terminal (F12) as seguintes informações:

- A largura total da janela do navegador (usando o objeto **window**).
- O título atual da aba do site (usando o objeto **document**).

**Por que fazer isso?** Para entender que o JavaScript tem acesso tanto ao navegador quanto ao conteúdo da página.

ATIVIDADE

# Portal EcoCycle

2

## Seleção e Mudança de Texto

**Contexto: Portal EcoCycle (Notícias de Ecologia)**

Utilize o **document.querySelector** para selecionar o título principal do portal (**#titulo-portal**).

- Altere o texto original para: **"EcoCycle - Notícias Sustentáveis"** usando **textContent**.
- Mude a cor desse título para um tom de verde usando o método de estilo **.style**.

ATIVIDADE

# Portal EcoCycle

## 3 Seleção por Posição e Manipulação de Classe

**Contexto: Portal EcoCycle (Notícias de Ecologia)**

Selecione o primeiro card de notícia da lista. Em vez de usar um ID, utilize o seletor de posição: **document.querySelector('.card-noticia:nth-of-type(1)')**.

- Adicione a este card a classe CSS **.noticia-destaque** utilizando o comando **classList.add**.
- *Nota: Verifique se a classe já existe no seu arquivo CSS com um destaque visual.*

ATIVIDADE

# Portal EcoCycle

4

## Manipulação Moderna de Atributos

**Contexto: Portal EcoCycle (Notícias de Ecologia)**

Selecione a imagem da thumbnail do vídeo através do ID **#thumb-video**.

- Atualize o atributo **alt** da imagem para: **"Clique para assistir sobre Energia Solar"**.
- Atualize o caminho da imagem (**src**) para uma nova imagem de sua pasta.
- **Importante:** Utilize a **versão moderna** de acesso direto (ex: **elemento.src = "caminho"**).

ATIVIDADE

# Portal EcoCycle

5

## Seleção de Coleções (querySelectorAll)

**Contexto: Portal EcoCycle (Notícias de Ecologia)**

Utilize o **document.querySelectorAll** para capturar todos os botões de leitura do portal (classe **.btn-leitura**).

- No **console.log**, exiba a frase: **"O portal EcoCycle possui [X] notícias disponíveis"**, onde [X] deve ser o número total de botões encontrados na lista.

# Referências

SENAI-SP. **Desenvolvimento de Sistemas: Front-end**. São Paulo: Editora SENAI, 2023. MDN WEB DOCS. **CSS Media Queries**. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/CSS/CSS_media_queries.

W3C. **Media Queries Level 4**. Disponível em: https://www.w3.org/TR/mediaqueries-4/.

MARCOTTE, Ethan. **Responsive Web Design**. A Book Apart, 2011.

FOTO DA ESCOLA

## ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

**Telefone**

(11) 2396-1999

**Instagram**

@senai.itu

**Facebok**

/senaisp.itu

**Site**

https://sp.senai.br/unidade/itu/